Лабораторная работа №16

Выбор и обоснование средств реализации проекта для разрабатываемой программной системы.

Задание 1. Обоснование выбора языка программирования

Для разработки игры «Восстание машин» выбран язык C++ (стандарт C++17) по следующим критериям:

- Высокая производительность и управление памятью: Игра требует стабильных 60 FPS, быстрого отклика UI (< 0.05 сек) и объема RAM < 1.5 ГБ. Ручное управление памятью и отсутствие «мусоросборщика» исключают фризы.
- Поддержка объектно-ориентированного программирования (ООП): Наличие классов позволяет удобно структурировать сущности (герой, роботы, инвентарь, локации).
- Совместимость с мультимедийными библиотеками: Отличная интеграция с графической библиотекой SFML 2.6.2 для отрисовки 2D-графики, обработки ввода и воспроизведения пространственного аудио.

Задание 2. Сравнительная характеристика парадигм программирования

№	Название парадигмы	Сущность парадигмы	Языки	Достоинства	Недостатки	Примеры программ
1	Императивное	Описание пошагового алгоритма изменения состояния программы	C, Pascal, Fortran	Прямой контроль над потоком выполнения и памятью	Сложность поддержки при увеличении объема кода	Калькуляторы, драйверы
2	Декларативное	Описание конечного результата (что получить), а не способа его достижения	SQL, HTML, Prolog	Высокий уровень абстракции, лаконичность	Меньше возможностей для оптимизации производительности	Запросы к БД, верстка страниц

3	Структурное	Разбиение программы на блоки с использованием ветвлений, циклов и подпрограмм (без goto)	C, Pascal, Ada	Понятная логика, легкость чтения и отладки	Ограниченная масштабируемость для сложных объектов	Алгоритмы сортировки
4	Процедурное	Группировка кода в многократно используемые процедуры и функции	C, Pascal, Basic	Модульность, повторное использование кода	Сложно связывать данные и функции в крупных проектах	Вычислительные утилиты
5	Модульное	Разделение кода на независимые файлы-модули с четким интерфейсом	Modula-2, C, Python	Удобство командной разработки, изолированность	Требует тщательного проектирования архитектуры	Системные библиотеки
6	Объектно-ориентированное	Программа строится из объектов, объединяющих данные (поля) и поведение (методы)	C++, Java, C#	Высокая повторная используемость, моделирование реальных сущностей	Избыточность кода, затраты ресурсов на абстракции	Игровые движки, GUI-приложения
7	Функциональное	Вычисления как вычисление математических функций	Haskell, Lisp, Erlang	Надежность, простота параллелизации, отсутствие сайд-эффектов	Высокий порог вхождения, требовательность к памяти	Системы обработки потоков данных

		без изменения состояний (чистые функции)				
8	Логическое	Программа задается в виде фактов и правил логического вывода	Prolog, Datalog	Эффективно для задач искусственного интеллекта и поиска решений	Низкая производительность в задачах общего назначения	Экспертные системы, базы знаний

- Вывод о наиболее часто используемых парадигмах: Наиболее распространенными в индустрии являются объектно-ориентированная (ООП), процедурно-структурная и модульная парадигмы, а также их сочетание в рамках мультипарадигменных языков.
- Обоснование парадигмы для проекта «Восстание машин»: Для проекта используется мультипарадигменный подход на основе ООП и модульного программирования. ООП подходит для описания игровых объектов (Player, Robot, Item), модульный подход — для разделения графики, аудио и сохранения в JSON, а процедурные элементы — для обработки игрового цикла и физики.

Задание 3. Примеры листинга программ (C++)

1. Структурное/процедурное программирование (расчет урона по блоку):

```
// Функция: применяет урон и возвращает оставшееся здоровье
int applyDamage(int currentHP, int damage) {
    int newHP = currentHP - damage;     // Вычитаем урон
    return (newHP < 0) ? 0 : newHP;     // Здоровье не может быть меньше 0
}

// Использование:
int blockHealth = 100;          // Создаем блок со 100 HP
int bulletDamage = 15;          // Пуля наносит 15 урона
blockHealth = applyDamage(blockHealth, bulletDamage);  // blockHealth = 85

// Проверка:
if (blockHealth == 0) {
    std::cout << "Блок уничтожен!" << std::endl;
}
```

2. Модульное программирование (система сохранения):

```
// ========== МОДУЛЬ СОХРАНЕНИЯ ==========
// Сохранение игры в файл
void saveGame() {
    json saveData;                    // Создаем JSON объект
    saveData["money"] = money;          // Добавляем монеты
    saveData["damageLevel"] = damageLevel; // Добавляем уровень урона

    std::ofstream file("save.json");     // Открываем файл
    if (file.is_open()) {               // Если файл открыт
        file << saveData.dump(4);        // Записываем данные
        file.close();                   // Закрываем файл
    }
}

// Загрузка игры из файла
void loadGame() {
    std::ifstream file("save.json");     // Открываем файл
    if (file.is_open()) {               // Если файл существует
        json saveData;                  // Создаем JSON объект
        file >> saveData;               // Читаем данные
        file.close();                   // Закрываем файл

        // Восстанавливаем данные
```

```
        if (saveData.contains("money")) money = saveData["money"];
        if (saveData.contains("damageLevel")) damageLevel = saveData["damageLevel"];
    }
}
// ============ ИСПОЛЬЗОВАНИЕ ============
int main() {
    loadGame();  // Загружаем сохранение
    // Играем...
    money += 50;
    damageLevel++;
    saveGame();  // Сохраняем прогресс
    return 0;
}
```

3. Объектно-ориентированное программирование (класс врага):

```
// ============ КЛАСС ПУЛЯ ============
class Bullet {
public:
    sf::CircleShape shape;  // Форма пули
    int damage;             // Урон
    bool active;            // Активна ли

    // Конструктор - создает пулю
    Bullet(float x, float y, int dmg) {
        shape.setRadius(10);        // Размер
        shape.setPosition(x, y);    // Позиция
        damage = dmg;               // Урон
        active = true;              // Активна
    }

    // Обновление - двигает пулю вверх
    void update(float dt) {
```

```
        shape.move(0, -800 * dt);   // Летит вверх
    }

    // Отрисовка - рисует пулю на экране
    void draw(sf::RenderWindow& window) {
        if (active) window.draw(shape);
    }
};

// ============ ИСПОЛЬЗОВАНИЕ ============

std::vector<Bullet> bullets;              // Массив пуль
bullets.emplace_back(400, 500, 15);       // Создаем пулю

for (auto& b : bullets) {
    b.update(dt);    // Обновляем каждую пулю
    b.draw(window);  // Рисуем каждую пулю
}
```

Ответы на контрольные вопросы:

1. *Язык программирования* — формальная знаковая система, предназначенная для записи компьютерных программ. Алфавит — набор допустимых символов; синтаксис — правила построения конструкций языка; семантика — смысловое значение конструкций; стандарт — документ, описывающий точные спецификации языка. *Классификации:* по уровню абстракции (низкоуровневые, высокоуровневые), по парадигме (ООП, функциональные и др.), по способу исполнения (компилируемые, интерпретируемые).

2. Критерии выбора языка:

- *Быстродействие и эффективность:* скорость работы и использование памяти.
- *Безопасность и надежность:* контрольные механизмы против ошибок.
- *Скорость разработки:* наличие удобного синтаксиса и экосистемы.
- *Поддержка библиотек:* availability фреймворков под задачу (например, SFML для GUI/аудио).
- *Квалификация команды:* опыт разработчиков в конкретном языке.

3. Парадигма программирования — совокупность идей, принципов и концепций, определяющих стиль написания программного кода.

4. *(Дана характеристика 8 парадигмам в таблице Задания 2).*

5. Стандарт оформления кода — свод правил и соглашений по написанию и форматированию исходного кода. Необходим для улучшения читаемости, облегчения отладки и поддержки кода в команде. *Правила:*

единый стиль отступов, правила именования, лимит длины строк, использование комментариев.

6. Объяснение правил оформления:

- *Венгерская нотация:* добавление к имени переменной префикса, отражающего ее тип (например, iHealth для int, strName для string).
- *Верблюжья нотация (CamelCase):* написание составных слов без пробелов, где каждое новое слово начинается с заглавной буквы (currentHealth, calculateDamage).
- *Грамотный выбор идентификаторов:* имена должны быть понятными и отражать суть сущности; константы записываются верхним регистром (MAX_HP).
- *Отступы:* использование 4 пробелов или табуляции для визуального выделения вложенных блоков кода.
- *Комментарии:* пояснения к сложным алгоритмам и функциям без дублирования очевидного кода.
- *Разбиение задачи на подзадачи:* декомпозиция крупного монолитного кода на небольшие изолированные функции и модули.
- *Правила оформления процедур, функций, модулей и классов:* логическое группирование связанных функций в классы/модули, соблюдение принципа одной ответственности (SRP), вынос объявлений в заголовочные файлы (.h).